6.4

a.) What appear to be the three or four most important car specifications for predicting the car's price?

Answer: The most important car specifications are - Automatic air conditioning, Fuel type, Quarterly road tax, Automatic transmission.

In [1]:
```
pip install mlba
```

```
Requirement already satisfied: mlba in /opt/anaconda3/lib/python3.13/site-pa
ckages (2.0.2)
Note: you may need to restart the kernel to use updated packages.
```

In [2]:
```
pip install mlxtend
```

```
Requirement already satisfied: mlxtend in /opt/anaconda3/lib/python3.13/site
-packages (0.23.4)
Requirement already satisfied: scipy>=1.2.1 in /opt/anaconda3/lib/python3.1
3/site-packages (from mlxtend) (1.15.3)
Requirement already satisfied: numpy>=1.16.2 in /opt/anaconda3/lib/python3.1
3/site-packages (from mlxtend) (2.1.3)
Requirement already satisfied: pandas>=0.24.2 in /opt/anaconda3/lib/python3.
13/site-packages (from mlxtend) (2.2.3)
Requirement already satisfied: scikit-learn>=1.3.1 in /opt/anaconda3/lib/pyt
hon3.13/site-packages (from mlxtend) (1.6.1)
Requirement already satisfied: matplotlib>=3.0.0 in /opt/anaconda3/lib/pytho
n3.13/site-packages (from mlxtend) (3.10.0)
Requirement already satisfied: joblib>=0.13.2 in /opt/anaconda3/lib/python3.
13/site-packages (from mlxtend) (1.4.2)
Requirement already satisfied: contourpy>=1.0.1 in /opt/anaconda3/lib/python
3.13/site-packages (from matplotlib>=3.0.0->mlxtend) (1.3.1)
Requirement already satisfied: cycler>=0.10 in /opt/anaconda3/lib/python3.1
3/site-packages (from matplotlib>=3.0.0->mlxtend) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in /opt/anaconda3/lib/pytho
n3.13/site-packages (from matplotlib>=3.0.0->mlxtend) (4.55.3)
Requirement already satisfied: kiwisolver>=1.3.1 in /opt/anaconda3/lib/pytho
n3.13/site-packages (from matplotlib>=3.0.0->mlxtend) (1.4.8)
Requirement already satisfied: packaging>=20.0 in /opt/anaconda3/lib/python
3.13/site-packages (from matplotlib>=3.0.0->mlxtend) (24.2)
Requirement already satisfied: pillow>=8 in /opt/anaconda3/lib/python3.13/si
te-packages (from matplotlib>=3.0.0->mlxtend) (11.1.0)
Requirement already satisfied: pyparsing>=2.3.1 in /opt/anaconda3/lib/python
3.13/site-packages (from matplotlib>=3.0.0->mlxtend) (3.2.0)
Requirement already satisfied: python-dateutil>=2.7 in /opt/anaconda3/lib/py
thon3.13/site-packages (from matplotlib>=3.0.0->mlxtend) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /opt/anaconda3/lib/python3.1
3/site-packages (from pandas>=0.24.2->mlxtend) (2024.1)
Requirement already satisfied: tzdata>=2022.7 in /opt/anaconda3/lib/python3.
13/site-packages (from pandas>=0.24.2->mlxtend) (2025.2)
Requirement already satisfied: six>=1.5 in /opt/anaconda3/lib/python3.13/sit
e-packages (from python-dateutil>=2.7->matplotlib>=3.0.0->mlxtend) (1.17.0)
Requirement already satisfied: threadpoolctl>=3.1.0 in /opt/anaconda3/lib/py
thon3.13/site-packages (from scikit-learn>=1.3.1->mlxtend) (3.5.0)
Note: you may need to restart the kernel to use updated packages.
```

In [3]:

```
import matplotlib.pyplot as plt # for plotting
import mlba
import numpy as np # for numerical data handling
import pandas as pd # for tabular data handling
import statsmodels.formula.api as sm # for statistical modeling
from mlxtend.feature_selection import ExhaustiveFeatureSelector, SequentialF
from sklearn.linear_model import BayesianRidge, Lasso, LassoCV, LinearRegres
from sklearn.model_selection import cross_validate, train_test_split, KFold,
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder,StandardScaler
%matplotlib inline
from mlxtend.feature_selection import SequentialFeatureSelector as SFS
from sklearn.linear_model import LinearRegression
from sklearn.linear_model import RidgeCV, LassoCV
from sklearn.linear_model import LinearRegression
```

```
from sklearn.compose import ColumnTransformer
import seaborn as sns
```

```
no display found. Using non-interactive Agg backend
```

In [4]:
```
#VB67
# Loading dataset
df = mlba.load_data("ToyotaCorolla-original.csv")

# Outcome variable
y = df['Price']

# Predictor variables
X = df[['Age_08_04', 'KM', 'CC', 'Fuel_Type', 'HP', 'Automatic', 'Doors',
        'Quarterly_Tax', 'Mfr_Guarantee', 'Guarantee_Period', 'Airco',
        'Automatic_airco', 'CD_Player', 'Powered_Windows', 'Sport_Model', 'T
```

In [5]:
```
#VB67
#Showing all columns
df.columns
```

Out[5]:
```
Index(['Id', 'Model', 'Price', 'Age_08_04', 'Mfg_Month', 'Mfg_Year', 'KM',
       'Fuel_Type', 'HP', 'Met_Color', 'Color', 'Automatic', 'CC', 'Doors',
       'Cylinders', 'Gears', 'Quarterly_Tax', 'Weight', 'Mfr_Guarantee',
       'BOVAG_Guarantee', 'Guarantee_Period', 'ABS', 'Airbag_1', 'Airbag_
2',
       'Airco', 'Automatic_airco', 'Boardcomputer', 'CD_Player',
       'Central_Lock', 'Powered_Windows', 'Power_Steering', 'Radio',
       'Mistlamps', 'Sport_Model', 'Backseat_Divider', 'Metallic_Rim',
       'Radio_cassette', 'Parking_Assistant', 'Tow_Bar'],
      dtype='object')
```

In [6]:
```
#VB67
#Spliting dataset into training (60%) and holdout/test (40%) sets for model

X_train, X_holdout, y_train, y_holdout = train_test_split(
    X, y, test_size=0.4, random_state=42
)
```

In [7]:
```
#VB67
# ColumnTransformer applies different preprocessing steps to numeric and cat
# StandardScaler standardizes numeric features to have zero mean and unit va

categorical_cols = ['Fuel_Type', 'Automatic', 'Doors', 'Mfr_Guarantee', 'Air
                    'Automatic_airco', 'CD_Player', 'Powered_Windows', 'Spor

numeric_cols = ['Age_08_04', 'KM', 'CC', 'HP', 'Quarterly_Tax', 'Guarantee_P

# OneHotEncoder transforms categorical features into binary variables, dropp

preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numeric_cols),
        ('cat', OneHotEncoder(drop='first'), categorical_cols)
    ]
)
```

In [8]:
```
#VB67
# Pipeline for linear regression
lr_pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('model', LinearRegression())
])

# Fit model
lr_pipeline.fit(X_train, y_train)

# Training performance
train_pred = lr_pipeline.predict(X_train)
holdout_pred = lr_pipeline.predict(X_holdout)
```

b.) Using metrics you consider useful, assess the performance of the model in predicting prices. Use cross-validation to assess the model's performance.

In [9]:
```
#VB67
# This function computes regression metrics (RMSE, MAE, R2) to assess predic

from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_scor

# RMSE indicates error magnitude, MAE reflects average error, and R2 shows v

def evaluate(y_true, y_pred, label):
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(y_true, y_pred)
    r2 = r2_score(y_true, y_pred)
    print(f"{label} - RMSE: {rmse:.2f}, MAE: {mae:.2f}, R2: {r2:.3f}")

# printing results for both training and holdout sets for model comparison.
```

```
evaluate(y_train, train_pred, "Training")
evaluate(y_holdout, holdout_pred, "Holdout")
```

```
Training - RMSE: 1198.90, MAE: 908.56, R2: 0.893
Holdout - RMSE: 1219.91, MAE: 894.28, R2: 0.882
```

c.) Using metrics you consider useful, assess the performance of the model in predicting prices. Use cross-validation to assess the model's performance.

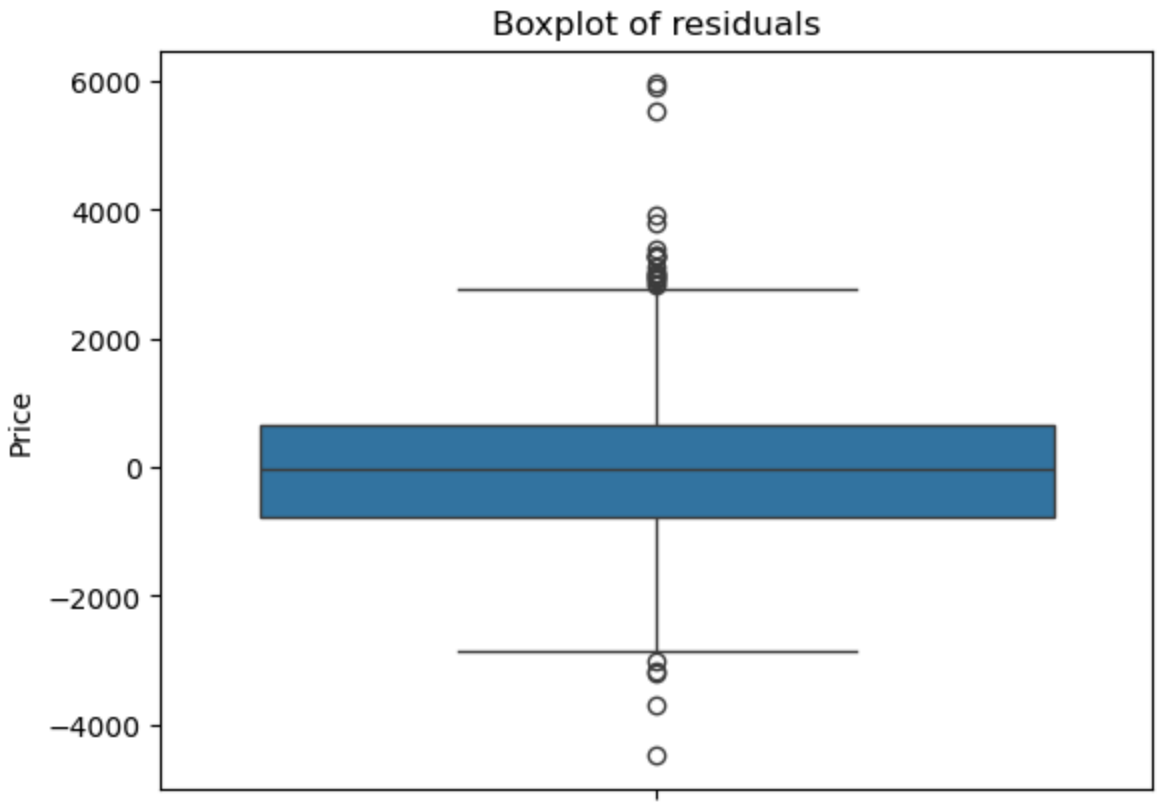
In [10]:

```
#VB67
# Performing 5-fold cross-validation on the training set using Linear Regres
# Evaluating model performance with RMSE and MAE
# Negatives are used because sklearn expects a "score" to be maximized

scoring = {'neg_RMSE': 'neg_root_mean_squared_error',
           'neg_MAE': 'neg_mean_absolute_error'}
scores = cross_validate(lr_pipeline,X_train, y_train, cv=5, scoring=scoring)

# Printing average cross-validated RMSE and MAE to assess model generalizati

print(f"CV-RMSE = {- scores['test_neg_RMSE'].mean():.2f}")
print(f"CV-MAE = {- scores['test_neg_MAE'].mean():.2f}")
```

```
CV-RMSE = 1887.05
CV-MAE = 1005.53
```

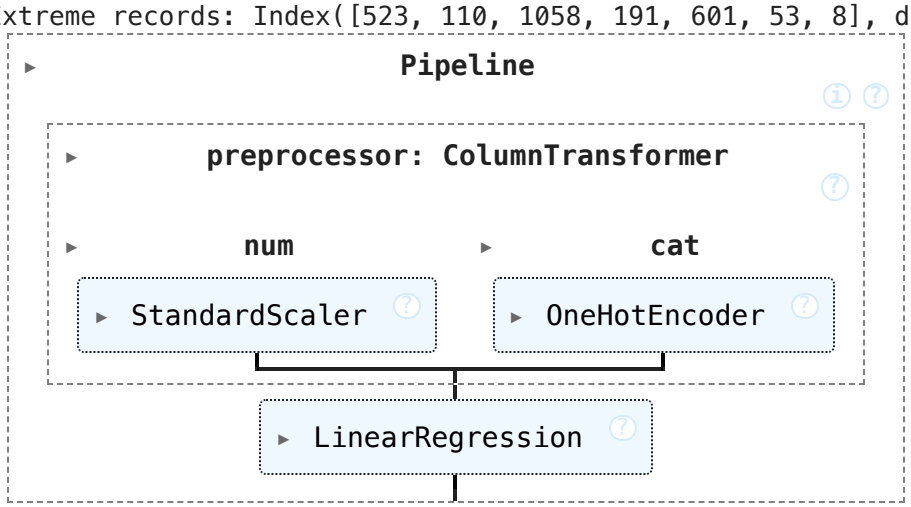
In [11]:

```
#VB67

# Residuals on training set
residuals = y_train - train_pred

# Visualizing residuals
sns.boxplot(residuals)
plt.title("Boxplot of residuals")
plt.show()

# Optionally remove extreme outliers
outlier_threshold = 3 * residuals.std()
outliers = residuals[abs(residuals) > outlier_threshold].index
print("Extreme records:", outliers)

# Removing outliers and refit
X_train_clean = X_train.drop(index=outliers)
y_train_clean = y_train.drop(index=outliers)
lr_pipeline.fit(X_train_clean, y_train_clean)
```




```
Extreme records: Index([523, 110, 1058, 191, 601, 53, 8], dtype='int64')
```

Out[11]:



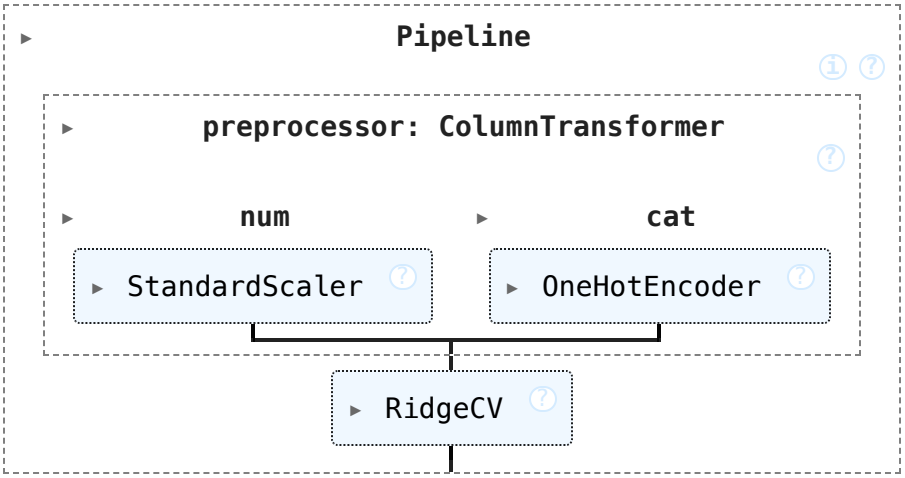
d.) Train lasso and ridge regression models, and compare their performance to the multiple linear regression model. Tune the Math equation parameter using 5-fold cross-validation for lasso and leave-one-out cross-validation for ridge regression (default for both methods).

In [12]:

```
#VB67
# Lasso with 5-fold CV
lasso_pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
```

```
    ('model', LassoCV(cv=5, random_state=42))
])
lasso_pipeline.fit(X_train_clean, y_train_clean)

# Ridge with leave-one-out CV
ridge_pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('model', RidgeCV(cv=None))  # default is leave-one-out
])
ridge_pipeline.fit(X_train_clean, y_train_clean)
```

Out[12]:



In [13]:

```
#VB67
# This loop evaluates and compares the performance of multiple regression mo
# The models dictionary maps model names to their respective trained pipelin

models = {
    'Linear Regression': lr_pipeline,
    'Lasso': lasso_pipeline,
    'Ridge': ridge_pipeline
}

# For each model, predictions are generated on both the training and holdout
for name, model in models.items():
    train_pred = model.predict(X_train_clean)
    holdout_pred = model.predict(X_holdout)
    print(f"\n{name} Performance:")
# This function helps in printing RMSE, MAE, and R2 metrics for each model,w
    evaluate(y_train_clean, train_pred, "Training")
    evaluate(y_holdout, holdout_pred, "Holdout")
```

```
Linear Regression Performance:
Training - RMSE: 1114.87, MAE: 864.82, R2: 0.902
Holdout - RMSE: 1232.20, MAE: 887.24, R2: 0.880

Lasso Performance:
Training - RMSE: 1131.93, MAE: 874.33, R2: 0.899
Holdout - RMSE: 1257.28, MAE: 898.99, R2: 0.875

Ridge Performance:
Training - RMSE: 1128.93, MAE: 869.61, R2: 0.900
Holdout - RMSE: 1256.21, MAE: 900.63, R2: 0.875
```

In [14]:
```
#VB67
# Extracting the feature names after preprocessing
preprocessor.fit(X_train_clean)
num_features = numeric_cols
cat_features = preprocessor.named_transformers_['cat'].get_feature_names_out
all_features = np.concatenate([num_features, cat_features])

# Coefficients from linear regression
coeffs = lr_pipeline.named_steps['model'].coef_
feature_importance = pd.DataFrame({'Feature': all_features, 'Coefficient': c
feature_importance = feature_importance.reindex(coeffs.argsort()[::-1])
# sorted by absolute value
# printing the results
print(feature_importance.head(10))
```

```
              Feature  Coefficient
14  Automatic_airco_1  2885.300530
7    Fuel_Type_Petrol  1738.044763
6    Fuel_Type_Diesel  1613.143298
4       Quarterly_Tax   639.713599
8         Automatic_1   603.668061
16  Powered_Windows_1   478.060439
17      Sport_Model_1   344.523589
3                  HP   331.354945
15        CD_Player_1   327.645100
13            Airco_1   244.102299
```

In [15]:
```
#VB67
# After fitting lr_pipeline on clean training data
preprocessor.fit(X_train_clean)
# Getting the names of the numeric features
num_features = numeric_cols
# Getting the names of one-hot encoded categorical features generated by the
cat_features = preprocessor.named_transformers_['cat'].get_feature_names_out

# Combining numeric and categorical feature names into a single array
all_features = np.concatenate([num_features, cat_features])

# Extracting the coefficients of the linear regression model from the pipeli
coeffs = lr_pipeline.named_steps['model'].coef_

# Creating a DataFrame associating each feature with its coefficient
feature_importance = pd.DataFrame({'Feature': all_features, 'Coefficient': c

# Adding a column for the absolute value of coefficients for sorting purpose
feature_importance['AbsCoeff'] = feature_importance['Coefficient'].abs()

# Sorting features by absolute coefficient value in descending order to iden
feature_importance = feature_importance.sort_values(by='AbsCoeff', ascending

# Displaying the top 10 features by their absolute coefficient importance
feature_importance.head(10)
```

Out[15]:

	Feature	Coefficient	AbsCoeff
14	Automatic_airco_1	2885.300530	2885.300530
0	Age_08_04	-2110.230539	2110.230539
7	Fuel_Type_Petrol	1738.044763	1738.044763
6	Fuel_Type_Diesel	1613.143298	1613.143298
10	Doors_4	-771.666210	771.666210
9	Doors_3	-752.715849	752.715849
1	KM	-683.169100	683.169100
4	Quarterly_Tax	639.713599	639.713599
8	Automatic_1	603.668061	603.668061
16	Powered_Windows_1	478.060439	478.060439

e) Compare the training, cross-validation, and holdout performance for all three models. Which model would you select based on the results and why?

Answer: Since the three models performed relatively similar, Linear regression did effectively performed better in training and holdout performance i.e., RMSE = 1232, R^2 = 0.880. Lasso and Ridge regularizations doesn't make any meaningful improvements since the dataset seems to be free from multicollinearity and overfitting. Therefore, Linear Regression is the idea choice as it has lowest prediction error and is easy to Interpret.